

DefiLlama Integration Guide: A Project-Agnostic Reference for Agentic AI Systems

TL;DR

- **DefiLlama is the de-facto open-source DeFi data plane** (TVL, yields, prices, stablecoins, fees, volumes, bridges, unlocks, hacks, raises) exposed through a free REST API at `api.llama.fi` and a paid Pro/API tier at `pro-api.llama.fi/{KEY}` priced at **\$300/month for 1,000 requests/min and 1 million calls/month** (overage \$0.60 / 1,000 calls); integrate via direct REST for backend services, via the official MCP server at `https://mcp.defillama.com/mcp` for agent-native consumption, and via the LlamaSwap iframe widget or its source-available adapter codebase `(github.com/LlamaSwap/interface)` for execution. `(llama)`
- **For a CEO-plus-seven-counsellors agent topology**, the natural partition is: Treasury (stablecoins+TVL), Yield (/pools), Risk (hacks/unlocks/oracles), Execution (LlamaSwap routing), Market (prices/volumes), Protocol-Intel (fees/revenue/users), Cross-Chain (bridges), and Macro (derivatives/OI/ETFs) — each Counsellor gets a curated tool subset on the same Pro API credit pool, with the CEO retaining sole access to write-side execution tools.
- **The single biggest design decision** is whether to consume DefiLlama through (a) direct REST with server-side caching for deterministic backend workflows, or (b) the official MCP server for agent-native natural-language access; production systems should run both — REST for hot paths and dashboards, MCP for ad-hoc agent reasoning — sharing a single API key.

1. Platform Overview

1.1 What DefiLlama Is

DefiLlama is the largest free DeFi analytics aggregator in the crypto industry. It tracks Total Value Locked (TVL), revenue, fees, volume, and yields across 7,000+ protocols on 500+ chains, with the design goal of "transparent crypto data without ads." It launched in **October 2020** as a TVL tracker for Ethereum DeFi and grew into the industry's de-facto economic-data backbone — "most 'TVL' numbers cited elsewhere on the internet are DeFiLlama under the hood."

`(Datawallet + 3)`

The platform is operated by **Llama Corp**, an umbrella organisation; the pseudonymous developer **Oxngmi** is the primary lead and majority owner, with co-founders Charlie Watkins

and Ben Hauser. The methodology is open-source, the API is public, and the dashboards are free of charge with no login. [Datawallet + 2](#)

1.2 The Full Product Surface

Product	Surface	Purpose
DefiLlama (core)	defillama.com	TVL aggregator across protocols and chains
DefiLlama Yields	defillama.com/yields	Aggregated APY/TVL across thousands of pools
Stablecoins	defillama.com/stablecoins	Supply, dominance, peg, chain distribution
Fees & Revenue	defillama.com/fees	Protocol fees, revenue, earnings breakdown
DEX Volume	defillama.com/dexs	Spot DEX volume per chain and protocol
Derivatives / Perps	defillama.com/derivatives	Notional volume, open interest
Bridges	defillama.com/bridges	Cross-chain bridge volume and flows
Liquidations	defillama.com/liquidations	Liquidation feeds (read-only)
NFT Data	defillama.com/nfts	Floor prices, volumes
Unlocks / Emissions	defillama.com/unlocks	Token vesting schedules
Hacks	defillama.com/hacks	Historical exploits, classifications, amounts
Raises	defillama.com/raises	Funding rounds, investors, amounts
Treasuries	defillama.com/treasuries	Protocol treasury balances
ETFs / DAT	defillama.com/etfs	Crypto ETF flows; institutional treasury holdings
CEX Transparency	defillama.com/cexs	Centralised-exchange TVL, inflows, leverage
LlamaSwap	swap.defillama.com	Zero-fee meta-DEX aggregator
LlamaPay	llamapay.io	Non-custodial streaming payments / payroll
LlamaFolio	llamafolio.com	DeFi portfolio tracker (open-source)
LlamaNodes / LlamaRPC	llamanodes.com	Public + premium RPC endpoints

Product	Surface	Purpose
ChainList	chainlist.org	EVM network metadata for wallets
LlamaFeed	defillama.com/llamafeed	Real-time crypto feed
LlamaSearch	DefiLlama tools	Project-link directory & domain-safety extension
LlamaAI	Pro subscription	Built-in AI chat over DefiLlama data
DL News	dlnews.com	Editorial / news arm

1.3 Open Source, GitHub Organization, Licensing

The GitHub organization is (github.com/DefiLlama), with 47 public repositories. The frontend ([defillama-app](#)) and the EVM network list ([chainlist](#)) are explicitly licensed **GPL-3.0-or-later**. The adapter repos ([DefiLlama-Adapters](#), [dimension-adapters](#), [peggedassets-server](#), [yield-server](#), [bridges-server](#), [defillama-server](#)) are all public and contribution-driven; most lack an explicit SPDX file at the root but inherit the GPL-3.0 ethos through [defillama-app](#). [GitHub](#)

The LlamaSwap interface (github.com/LlamaSwap/interface) is publicly viewable but **does not appear to carry a formal SPDX license file** at the repository root — a notable point for downstream integrators who plan to vendor or fork it. Treat it as source-available rather than formally open-source; contact 0xngmi via the DefiLlama Discord before redistributing.

Governance is informal — a small core team of "llamas" reviews adapter PRs; community contributors maintain most protocol-specific adapters. The official Discord is the primary coordination channel.

2. Complete DefiLlama API Reference

2.1 Base URLs and Authentication

Service	Base URL	Auth	Notes
TVL/ protocols	https://api.llama.fi	None	Free tier
Coins/prices	https://coins.llama.fi	None	Free tier
Stablecoins	https://stablecoins.llama.fi	None	Free tier

Service	Base URL	Auth	Notes
Yields	<code>https://yields.llama.fi</code>	None for <code>/pools</code> ; Pro for some endpoints	
Bridges (list)	<code>https://bridges.llama.fi</code>	None (list); Pro for detail	
Pro API	<code>https://pro-api.llama.fi/{API_KEY}</code>	API key in path	\$300/month

Two important rules from the official docs:

1. The Free API and Pro API are **entirely separate services** — do not mix base URLs. [defillama](#)
2. Pro authentication is **API key in the URL path**, not in a header: `https://pro-api.llama.fi/{YOUR_API_KEY}/api/protocols`. [defillama](#)

2.2 TVL Endpoints ([api.llama.fi](#))

Endpoint	Purpose
<code>GET /protocols</code>	List all protocols with current TVL defillama
<code>GET /protocol/{protocol}</code>	Historical TVL of a protocol, breakdown by token and chain defillama
<code>GET /tvl/{protocol}</code>	Simplified current TVL of a protocol defillama (number)
<code>GET /v2/chains</code>	Current TVL of all chains defillama
<code>GET /v2/historicalChainTvl</code>	Historical TVL of all chains (excludes liquid staking + double counted) defillama
<code>GET /v2/historicalChainTvl/{chain}</code>	Historical TVL of a single chain defillama
<code>GET /charts/{chain}</code>	Aggregate chart data per chain

2.3 Coins / Prices Endpoints ([coins.llama.fi](#))

Coin identifiers use the `{chain}:{address}` format, e.g.

`ethereum:0xdAC17F958D2ee523a2206206994597C13D831ec7`. Native tokens use `coingecko:{slug}` or chain-specific aliases. [GitHub](#)

Endpoint	Purpose
GET /prices/current/{coins}	Current prices, (defillama) comma-separated coin IDs
GET /prices/historical/{timestamp}/{coins}	Historical prices at a Unix timestamp (defillama)
GET /batchHistorical	Multi-coin, multi-timestamp historical fetch (defillama)
GET /chart/{coins}	Token price chart at regular intervals (defillama) (params: start, end, span, period, searchWidth)
GET /percentage/{coins}	% change over time (defillama)
GET /prices/first/{coins}	Earliest price record (defillama)
GET /block/{chain}/{timestamp}	Closest block to a timestamp (defillama)

2.4 Stablecoins Endpoints (stablecoins.llama.fi)

Endpoint	Purpose
GET /stablecoins	List all stablecoins with circulating amounts (defillama)
GET /stablecoincharts/all	Historical aggregate mcap of all stablecoins (defillama)
GET /stablecoincharts/{chain}	Historical mcap of stablecoins on a chain (defillama)
GET /stablecoin/{asset}	Historical mcap + chain distribution of a stablecoin (defillama)
GET /stablecoinchains	Current mcap sum per chain (defillama)
GET /stablecoinprices	Historical prices of all stablecoins (defillama)

2.5 Yields Endpoints (yields.llama.fi)

Endpoint	Purpose
<code>GET /pools</code>	All yield pools — returns <code>chain</code> , <code>project</code> , <code>symbol</code> , <code>tv1Usd</code> , <code>apy</code> , <code>apyMean30d</code> , <code>predictions</code> , <code>Glama</code> <code>pool</code> (UUID)
<code>GET /chart/{pool}</code>	Historical APY and TVL of a pool <code>defillama</code> by UUID

Pro-only yield endpoints: `/yields/poolsOld`, `/yields/poolsBorrow`, `/yields/chartLendBorrow/{pool}`, `/yields/perps`, `/yields/l1sdRates`. `defillama`

2.6 Fees, Revenue, and Volumes

Endpoint	Purpose
<code>GET /overview/fees</code>	All protocols' fees/revenue with history <code>defillama</code>
<code>GET /overview/fees/{chain}</code>	Same, filtered by chain <code>defillama</code>
<code>GET /summary/fees/{protocol}</code>	Per-protocol fees/revenue with history <code>defillama</code>
<code>GET /overview/dexs</code>	DEX spot volumes overview <code>defillama</code>
<code>GET /overview/dexs/{chain}</code>	DEX volumes filtered by chain <code>defillama</code>
<code>GET /summary/dexs/{protocol}</code>	Per-DEX volume summary <code>defillama</code>
<code>GET /overview/options</code>	Options DEX volumes <code>defillama</code>
<code>GET /overview/options/{chain}</code>	Same, by chain <code>defillama</code>
<code>GET /summary/options/{protocol}</code>	Per-protocol options summary <code>defillama</code>
<code>GET /overview/open-interest</code>	Open interest across perps DEXs <code>defillama</code>

Fee `dataType` enums (passed as `?dataType=`): `dailyFees`, `dailyRevenue`, `dailyUserFees`, `dailyHoldersRevenue`, `dailyCreatorRevenue`, `dailySupplySideRevenue`, `dailyProtocolRevenue`, `dailyBribesRevenue`, `dailyTokenTaxes`, `dailyAppFees`, `dailyAppRevenue`. `GitHub`

2.7 Bridges (free list, Pro detail)

Pro paths under `/bridges/`:

- `GET /bridges/bridges` — list with metadata
- `GET /bridges/bridge/{id}` — per-bridge summary
- `GET /bridges/bridgevolume/{chain}` — historical bridge volume per chain (defillama)
- `GET /bridges/bridgedaystats/{timestamp}/{chain}` — daily snapshot (defillama)
- `GET /bridges/transactions/{id}` — bridge transactions (defillama)

2.8 Pro-Exclusive Endpoints (`pro-api.llama.fi/{KEY}`)

- **Unlocks / Emissions:** `/api/emissions`, `/api/emission/{protocol}` (defillama)
- **Protocol Analytics:** `/api/categories`, `/api/forks`, `/api/oracles`, `/api/hacks`, `/api/raises`, `/api/treasuries`, `/api/entities`, `/api/inflows/{protocol}/{timestamp}`, `/api/tokenProtocols/{symbol}`, `/api/chainAssets` (defillama)
- **Stablecoin dominance:** `/stablecoins/stablecoindominance/{chain}` (defillama)
- **Token Liquidity:** `/api/historicalLiquidity/{token}` (defillama)
- **Yields (extended):** `/yields/poolsOld`, `/yields/poolsBorrow`, `/yields/chartLendBorrow/{pool}`, `/yields/perps`, `/yields/lldRates`
- **ETFs:** `/etfs/snapshot`, `/etfs/flows` (defillama)
- **DAT (Digital Asset Treasury):** `/dat/institutions`, `/dat/institutions/{symbol}` (defillama)
- **Narratives:** `/fdv/performance/{period}` (defillama)
- **Perpetuals / OI:** `/api/overview/derivatives`, `/api/summary/derivatives/{protocol}` (defillama)
- **Equities:** `/equities/v1/companies`, `/equities/v1/statements`, `/equities/v1/price-history`, `/equities/v1/ohlcv`, `/equities/v1/summary`, `/equities/v1/filings` (defillama)
- **API key management:** `/usage/APIKEY` (defillama)

2.9 Rate Limits, Pricing, Reliability

- **Free tier:** a publicly announced cap of **500 requests per minute** per IP. Per the official @DefiLlama X account on January 2, 2023: *"Starting tomorrow we'll impose a limit of 500 requests/min in our API. There's only a single party that is surpassing this limit currently, so there's no need to change anything if you're using our API, as you won't be impacted."* In practice the API is generous; respect 429s and back off. (X)
- **Pro / API tier:** \$300/month, with the official `docs.llama.fi/pro-api` page specifying exactly **1,000 requests per minute, 1 million API calls per month, \$0.60 per 1,000**

additional calls after the 1M cap. Includes the full set of 38 Pro-exclusive endpoints and the ability to call free endpoints on `pro-api.llama.fi` to inherit the higher limit. `llama`

- **DefiLlama contributors:** per the official upgrade page (`pro.llama.fi`), "DefiLlama contributors will have free 3 month access to premium API." `Llama`
- **Note:** The DefiLlama "Pro" subscription (LlamaAI + dashboards) and the "API" plan are **different products** — "API access is not included in the Pro plan." `GitHub`

2.10 Caching, Polling, Retry Best Practices

- **TVL updates:** roughly hourly; stablecoin data updates a few times per day; yields update hourly. Cache `/protocols`, `/v2/chains`, `/pools` for 5-10 minutes server-side at minimum. `Eco`
- **Prices:** `/prices/current/{coins}` is the highest-frequency endpoint; cache by token for 30-60 seconds for dashboards, longer for analytics.
- **Retry:** standard exponential backoff (0.5s → 1s → 2s → 4s, 5 attempts max) on 429 and 5xx. The official SDK exposes a typed `RateLimitError` with `retryAfter`. `GitHub`
- **Batching:** prefer `/batchHistorical` and comma-separated `/prices/current/{c1},{c2},{c3}` over per-token calls.

2.11 Official SDKs

Language	Package	Repo
TypeScript	<code>npm install @defillama/api</code> <code>defillama</code>	<code>github.com/DefiLlama/api-sdk</code> <code>defillama</code>
Python	<code>pip install defillama-sdk</code> <code>defillama</code>	<code>github.com/DefiLlama/python-sdk</code> <code>defillama</code>

Both SDKs accept an optional `apiKey` for Pro endpoints and expose typed errors (`ApiKeyRequiredError`, `RateLimitError`, `NotFoundError`, `ApiError`). `GitHub`

2.12 OpenAPI Specifications

DefiLlama publishes two OpenAPI 3.0 specs that can be ingested by tools like Postman, Swagger UI, or auto-generated clients:

- `https://api-docs.defillama.com/defillama-openapi-free.json` `defillama`
- `https://api-docs.defillama.com/defillama-openapi-pro.json` `defillama`

There are also two LLM-discovery files:

- <https://api-docs.defillama.com/llms-free.txt> [defillama](#)
 - <https://api-docs.defillama.com/llms-pro.txt> [defillama](#)
-

3. DefiLlama AI / MCP Integration

3.1 The Official MCP Server

DefiLlama operates an **official, hosted Model Context Protocol server** at

<https://mcp.defillama.com/mcp>. It uses **Streamable HTTP transport** (the current MCP spec) and falls back to SSE for stateful sessions. Key facts:

- **Auth model:** OAuth — your agent prompts the user to log in via browser to their DefiLlama account on first use. Token refreshes every 24 hours. [DefiLlama](#) [githubusercontent](#)
- **Subscription requirement:** an active **DefiLlama API plan** is required; the MCP uses the **same credit pool as your API key** (one credit per query, no separate billing). [DefiLlama](#)
- **Per-account binding:** each DefiLlama account can be connected to **only one MCP client at a time**; connecting a new client disconnects the previous one. [DefiLlama](#)

3.2 Connecting from a Client

Claude Code:

```
bash

claude mcp add defillama --transport http https://mcp.defillama.com/mcp
```

Claude Desktop / Cursor / Windsurf / Codex / Gemini CLI / OpenCode:

```
json

{
  "mcpServers": {
    "defillama": { "url": "https://mcp.defillama.com/mcp" }
  }
}
```

Stdio-only agents (use the [mcp-remote](#) bridge):

```
json
```

```

{
  "mcp": {
    "servers": {
      "defillama": {
        "command": "npx",
        "args": ["-y", "mcp-remote", "https://mcp.defillama.com/mcp"]
      }
    }
  }
}

```

3.3 The 24 Exposed Tools

Tool	Description
<code>resolve_entity</code>	Fuzzy-match protocol/chain/token names to canonical slugs (githubusercontent)
<code>get_market_totals</code>	Global DeFi TVL, DEX volume, derivatives volume (githubusercontent)
<code>get_protocol_metrics</code>	TVL, fees, revenue, mcap, ratios, trends (githubusercontent)
<code>get_protocol_info</code>	Metadata, URLs, audits, tags (githubusercontent)
<code>get_chain_metrics</code>	Chain TVL, gas, revenue, DEX volume (githubusercontent)
<code>get_chain_info</code>	Chain metadata, L2 parent (githubusercontent)
<code>get_category_metrics</code>	Category rankings (githubusercontent)
<code>list_categories</code>	All valid categories (githubusercontent)
<code>get_token_prices</code>	Token price, mcap, volume, ATH (githubusercontent)
<code>get_token_tvl</code>	Token deposits across protocols (githubusercontent)
<code>get_token_unlocks</code>	Vesting schedules and upcoming unlocks (githubusercontent)
<code>get_yield_pools</code>	Pool APY, TVL, lending/borrowing rates (githubusercontent)
<code>get_stablecoin_supply</code>	Stablecoin issuance by chain (githubusercontent)
<code>get_bridge_flows</code>	Bridge volume and net flows (githubusercontent)
<code>get_etf_flows</code>	BTC / ETH ETF flows (githubusercontent)

Tool	Description
<code>get_dat_holdings</code>	Institutional holdings + mNAV <code>(githubusercontent)</code>
<code>get_events</code>	Hacks, fundraises, protocol events <code>(githubusercontent)</code>
<code>get_oracle_metrics</code>	Oracle TVS and coverage <code>(githubusercontent)</code>
<code>get_cex_volumes</code>	CEX trading volume <code>(githubusercontent)</code>
<code>get_open_interest</code>	Derivatives OI <code>(githubusercontent)</code>
<code>get_treasury</code>	Protocol treasury holdings <code>(githubusercontent)</code>
<code>get_user_activity</code>	DAU, transactions <code>(githubusercontent)</code>
<code>get_income_statement</code>	Revenue breakdown <code>(githubusercontent)</code>
<code>get_my_usage</code>	Remaining API credits <code>(githubusercontent)</code>

3.4 Companion Workflow Skills

DefiLlama publishes **10 structured workflow skills** at `(github.com/DefiLlama/defillama-skills)` — these are prompt-engineered patterns that turn raw tool access into guided research workflows:

`(defi-data)`, `(defi-market-overview)`, `(protocol-deep-dive)`, `(token-research)`, `(chain-ecosystem)`, `(market-analysis)`, `(yield-strategies)`, `(risk-assessment)`, `(flows-and-events)`, `(institutional-crypto)`, `(githubusercontent)`

Install (auto-detects your agent):

```
bash
npx skills add DefiLlama/defillama-skills --yes
```

3.5 Community MCP Servers

If you need self-hosted control (rate-limit isolation, no per-user OAuth, custom tool surface), several open-source MCP servers wrap the free DefiLlama API:

- `(dcSpark/mcp-server-defillama)` — `(npx @mcp-dockmaster/mcp-server-defillama);stdio;`
tools: `(defillama_get_protocols)`, `(defillama_get_protocol_tv1)`,
`(defillama_get_chain_tv1)`, `(defillama_get_token_prices)`,

`defillama_get_historical_prices`, `defillama_get_stablecoins`,
`defillama_get_stablecoin_data`.

- `IQAIcom/mcp-defillama` — `pnpm dlx @iqai/defillama-mcp`; adds AI-powered auto-resolution for protocol/chain/stablecoin names; supports an optional `DEFILLAMA_API_KEY`; broader tool surface (chains, historical, pool, batch historical, percentage, chart). [GitHub](#)
- `nic0xflame1/defillama-mcp-server` — `npx -y @nic0xflame1/defillama-mcp-server`; OpenAPI-driven, dynamic tool generation. [GitHub](#)
- `demcp/demcp-defillama-mcp` — Python/FastMCP, Docker-shippable, supports SSE. [GitHub](#)
- **PulseMCP "Pipeworx DefiLlama"** — hosted, no-auth remote with Streamable HTTP, free.

3.6 Agent Consumption Patterns

LLM agents should:

1. **Always call `resolve_entity` first** when a user mentions a protocol/chain/token name — DefiLlama uses slugs (`aave-v3`, `lido`) that often differ from common names.
2. **Use the `/protocols` endpoint as a directory**, not as a data source; fetch the slug, then pull `/protocol/{slug}` for detail.
3. **Treat TVL as USD-denominated point-in-time**, not flow. For flows, combine with `/v2/historicalChainTvl/{chain}` deltas.
4. **For prices, prefer `/prices/current/{coins}` over `/chart/{coins}`** for single-shot questions; reserve `/chart` for trend analysis.
5. **Yields require post-filtering** — `/pools` returns ~20,000 records; an agent should request server-side caching and apply chain/project/APY/TVL filters client-side.

Example tool-call patterns:

User: "Compare Aave v3's TVL on Arbitrum vs Base over the last 90 days."

Agent:

1. `resolve_entity("Aave v3") -> "aave-v3"`
2. `get_protocol_metrics(protocol="aave-v3")` // returns chain breakdown
3. (optional) `GET /protocol/aave-v3` // for full historical
4. Compute Arbitrum and Base series; respond with comparison.

User: "What are the best USDC yields over \$10M TVL with APY > 6%?"

Agent:

1. `get_yield_pools(symbol="USDC")`
2. Filter `tv1Usd > 1e7` and `apy > 6`
3. Sort by `apyMean30d desc`
4. Cross-check `predictions.predictedClass` for confidence

4. LlamaSwap Deep Dive

4.1 Product Summary

LlamaSwap (swap.defillama.com) is DefiLlama's **zero-fee meta-DEX aggregator**: it queries multiple DEX aggregators in parallel, normalises their quotes for gas costs, and lets the user pick the best execution. Founder 0xngmi publicly emphasises that it "**hard-blocks high-risk trades that other platforms merely warn about**" — the swap button is disabled when price impact crosses a configurable threshold. [Pages](#) [Datawallet](#)

Key features:

- **Zero platform fees**; only network gas. [Pages](#)
- **Privacy "Hide IP" toggle** that proxies aggregator requests through DefiLlama servers to prevent aggregators from linking wallet addresses to IPs. [DL News](#)
- **40% gas-limit inflation** on transactions to absorb unexpected gas spikes; unused gas is refunded by the network. [COIN360](#)
- **Approval streamlining** — remembers which aggregator's router contracts the user has already approved. [COIN360](#)
- **Airdrop preservation** — quotes that go through a DEX that runs an airdrop program are tagged with a gift-box icon to signal eligibility retention. [COIN360](#)
- **22+ EVM chains** at launch, has since expanded; Solana is not yet supported. [Crypto News](#)

4.2 The Open-Source Repository

The frontend is github.com/LlamaSwap/interface — a Next.js / TypeScript app. Architecture:

- [src/components/Aggregator/adapters/](#) — one subdirectory per integrated aggregator.
- [src/components/Aggregator/constants.ts](#) — [chainsMap](#) mapping LlamaSwap chain names to numeric EVM chain IDs.
- [src/components/Aggregator/types.ts](#) — shared [ExtraData](#) and quote types.
- [server/](#) — backend helpers (gas estimation, multicall, etc.).

License: The repository **does not appear to carry a formal SPDX LICENSE file** at the root. The code is publicly viewable but legally source-available rather than formally open-source.

Downstream integrators planning to vendor or fork should contact 0xngmi via Discord before redistributing.

4.3 Supported Aggregators (confirmed adapters)

Each subdirectory of `src/components/Aggregator/adapters/` wraps a single underlying aggregator. Confirmed adapters include:

`cowswap`, `1inch`, `0x` (Matcha), `paraswap` (now Velora), `kyberswap`, `odos`, `openocean`, `yield-yak`, `llamazip` (DefiLlama's own zip aggregator), `firebird`, `hashflow`, `unindex`, plus a pending `conveyor` adapter (PR #246). The LlamaSwap protocol page on defillama.com lists active routes including **1inch**, **Hashflow**, **OpenOcean**, **Velora (ParaSwap)**, **Unizen**, **Houdini Swap**, **Pond**, **0x**, and **THORSwap**. `COIN360 + 2`

Note: A definitive directory listing of the adapters folder could not be retrieved during research. The list above is assembled from direct file fetches, merged PRs, and external coverage; it is close to but not guaranteed to be complete.

4.4 The Adapter Interface

Every aggregator adapter is a TypeScript module that exports a uniform interface (inferred from `cowswap/index.ts`):

```
typescript
```

```

// metadata
export const chainToId: Record<string, number | string>; // LlamaSwap chain → aggregator
export const name: string; // display name ("CowSwap")
export const token: string | null; // aggregator's governance token
export const referral: boolean; // referral revenue support
export const isOutputAvailable: boolean; // supports exact-output (no slippage)

// functions
export function approvalAddress(): string; // ERC-20 spender to approve

export async function getQuote(
  chain: string,
  from: string,
  to: string,
  amount: string,
  extra: ExtraData // { userAddress, slippage }
): Promise<{
  amountReturned: string; // expected output
  amountIn: string; // actual input (incl. fees)
  estimatedGas: number;
  validTo?: number; // order expiry, off-chain orders
  rawQuote: unknown; // opaque, passed back to swap()
  tokenApprovalAddress: string;
  logo?: string;
  isMEVSafe?: boolean;
}>;

export async function swap(args: {
  chain: string;
  fromAddress: string;
  rawQuote: unknown;
  from: string;
  to: string;
  isSmartContractWallet?: boolean;
}): Promise<string | { id: string; waitForOrder: () => Promise<any> }>;

export function getTxData(): string; // calldata, "" for off-chain
export function getTx(): { to?: string; data?: string; value?: string };

```

This uniform shape is what lets LlamaSwap fan out parallel quote requests to N aggregators and compare them on `amountReturned - estimatedGas * gasPrice`.

4.5 Meta-Aggregation Algorithm (Conceptual)

1. **Token list resolution:** `from` and `to` must be present in DefiLlama's token lists (anti-scram filter). [GitHub](#)
2. **Parallel quote fan-out:** For every adapter whose `chainToId` covers the requested chain, call `getQuote()` concurrently.
3. **Gas normalisation:** Multiply each adapter's `estimatedGas` by the current chain gas price (LlamaSwap maintains a custom price API for wide token coverage) to compute USD cost.
4. **Comparison:** Rank quotes by `amountReturned * priceOf(to) - gasCostUSD`.
5. **Display:** Surface the best route, but show the full sorted list — the user can pick a non-top route if they prefer a specific aggregator (e.g., MEV-safe CowSwap).
6. **Approval flow:** If the user has not approved `tokenApprovalAddress` for the spender of the chosen route, the UI requests an approve tx first.
7. **Execution:** Calls `swap()` on the chosen adapter — for CowSwap this signs an EIP-712 order and POSTs it; for on-chain aggregators it returns calldata that the wallet sends as a tx.

[GitHub](#)

4.6 Integration Surfaces

Three documented options:

1. **iframe widget** (preferred for most projects):

html

```
<iframe title="LlamaSwap Widget"
  src="https://swap.defillama.com?chain=ethereum"
  width="450" height="565"
  allow="fullscreen" frameborder="0"></iframe>
```

Customisable via query params: `chain` (required), `from`, `to` (must be in DefiLlama's token list; `0x000...` for the native gas token), `background`. [GitHub](#)

2. **API access:** Contact `@0xngmi` on the DefiLlama Discord for an API key. *"We are forced to use api keys because many of the underlying aggregators have rate limits, so we have to control the volume of requests we send to them."* [GitHub](#) [GitHub](#)
3. **Forking the repo:** Vendor `LlamaSwap/interface` and embed it directly. Be aware of the licensing caveat in §4.2.

4.7 Fee Structure

LlamaSwap charges **no platform or routing fees**. The launch tweet explicitly listed "no fees" as a headline feature. Underlying aggregators may charge their own fees (CowSwap solver fees, Ox integrator fees set to zero by LlamaSwap, etc.) — those are embedded in the quote. [Pages](#)

4.8 Reference Implementations

For server-side embedding, see how other projects extracted the routing logic:

- **AstrolabDAO/swapper** (github.com/AstrolabDAO/swapper) — explicitly "*inspired by LlamaSwap's work, supercharged with cross-chain capacity*"; a clean Swapper SDK that wraps 1inch, Ox, KyberSwap, ParaSwap, Li.Fi, Squid Router, Socket. [GitHub](#)
 - **bgd-labs/llamaswap-interface** — a fork by BGD Labs (the Aave Companies' delivery firm) used for embedding into the Aave frontend.
 - **Openspace-Protocol/swap-interface** — a fork by OpenSpace.
-

5. Integration Architecture Patterns

5.1 Pattern A — Direct REST Consumption (Server-Side)

```
[Cron / scheduled worker] → [Redis cache layer] → [App services]
                             ↳ api.llama.fi / pro-api.llama.fi
```

Use when you need deterministic data refresh, custom transformation, or feed downstream pipelines (Postgres warehouses, dashboards). Cron intervals: prices 30 s, TVL 5-10 min, yields 10-15 min, stablecoins hourly.

5.2 Pattern B — MCP-Native Agent Consumption

```
[Agent (LLM)] → MCP client (Claude Code / Cursor) → mcp.defillama.com/mcp
                                                         ↳ 24 typed tools
```

Use when the consumer is a natural-language agent. Pros: zero glue code, schema discovery, OAuth-managed credentials. Cons: per-account single-client limit; one credit per query (cost adds up at high agent volume); MCP not yet ideal for batch or streaming workloads.

5.3 Pattern C — Embedded Swap UI (iframe or fork)

Default: iframe swap.defillama.com for fastest integration. Advanced: fork

[LlamaSwap/interface](#), strip unused chains, replace branding, and rebuild. Beware the licensing

caveat.

5.4 Pattern D — Backend Swap Routing

Extract LlamaSwap's adapter pattern into a server-side service. The cleanest reference is [AstrolabDAO/swapper](#), whose [ISwapperParams](#) interface mirrors the LlamaSwap adapter contract:

```
typescript

interface ISwapperParams {
  aggregatorId: AggregatorId;
  inputChainId: number;
  outputChainId?: number; // cross-chain
  input: string;           // token addr
  output: string;
  amountWei: number;
  payer: `0x${string}`;
}
```

Use for: programmatic rebalancers, treasury bots, MEV-aware execution where you want to retain the routing logic but execute through your own RPC and wallet stack.

5.5 Pattern E — Hybrid Analytics + Execution

The canonical pattern for an agentic financial system:

```
TVL / Yields / Risk feeds (api.llama.fi) → Opportunity discovery
    ↓
Agent reasoning (mcp.defillama.com)      → Filter and rank
    ↓
LlamaSwap backend router (Pattern D)     → Execute trade
    ↓
On-chain confirmation                    → Update state
```

This is what an "AI portfolio manager" looks like in 2026: read-heavy DefiLlama on the analytics side, LlamaSwap-style multi-route execution on the action side.

5.6 Webhooks / Push

DefiLlama does **not** expose first-party webhooks. For push semantics, either (a) poll-and-diff at the application layer, (b) subscribe to LlamaFeed for human-readable events, or (c) hook into the GitHub adapter repos ([DefiLlama-Adapters](#), [dimension-adapters](#)) to be notified when a protocol changes methodology.

5.7 Community Libraries

Language	Library	Status
TypeScript	@defillama/api	Official
Python	defillama-sdk	Official
Python	DeFiLlama (PyPI, Apache 2.0, by itzmestar) PyPI	Community, no API key required PyPI
TypeScript	defillama-api (boulderbytes/HatiOx)	Community wrapper

6. Agent-Hierarchy Specific Guidance: CEO + Seven Counsellors

For a CEO-plus-seven-counsellors topology, the natural mapping is by financial *domain*, with each Counsellor receiving a curated subset of DefiLlama tools. The CEO retains coordination authority and exclusive access to write-side (LlamaSwap execution) tools.

6.1 Suggested Counsellor Mapping

Counsellor	Domain	DefiLlama tool subset (MCP names)	REST endpoints
Treasury	Stablecoins, chain TVL	get_stablecoin_supply, get_market_totals, get_chain_metrics, get_treasury	/stablecoins, /stablecoinchains, /v2/chains, /treasuries (Pro)
Yield	APY hunting, pool selection	get_yield_pools, get_token_tvl	/pools, /chart/{pool}, /yields/lsdRates (Pro)
Risk	Hacks, oracles, unlocks	get_events, get_oracle_metrics, get_token_unlocks	/hacks, /oracles, /emissions (all Pro)
Execution	Swap routing	(none MCP-side, by design)	LlamaSwap iframe or backend router

Counsellor	Domain	DefiLlama tool subset (MCP names)	REST endpoints
Market	Prices, DEX volumes	<code>get_token_prices</code> , <code>get_cex_volumes</code> , <code>resolve_entity</code>	<code>/prices/current/*</code> , <code>/overview/dexs</code>
Protocol-Intel	Fees, revenue, users	<code>get_protocol_metrics</code> , <code>get_protocol_info</code> , <code>get_income_statement</code> , <code>get_user_activity</code>	<code>/overview/fees</code> , <code>/summary/fees/*</code> , <code>/protocol/*</code>
Cross-Chain	Bridges	<code>get_bridge_flows</code>	<code>/bridges/*</code> (Pro for detail)
Macro	Derivatives, OI, ETFs, DAT	<code>get_open_interest</code> , <code>get_etf_flows</code> , <code>get_dat_holdings</code> , <code>get_category_metrics</code>	<code>/overview/derivatives</code> (Pro), <code>/etfs/*</code> (Pro)

6.2 CEO-Only Tool Surface

The CEO agent should hold:

- All **write-side** capabilities (LlamaSwap execution, on-chain tx signing).
- The shared `get_my_usage` tool — for global API-credit budgeting across Counsellors.
- `resolve_entity` as an arbiter when two Counsellors disagree on a slug.

6.3 Caching and Rate-Limit Budgeting

With a single \$300/month Pro key shared by 8 agents, the official ceiling (per `docs.llama.fi/pro-api`) is **1,000 requests/min** and **1 million calls/month**, giving a working ceiling of **60,000 requests/hour** and overage cost of \$0.60 per 1,000 calls beyond 1M/month:

Layer	Cache TTL	Notes
L1 (per-Counsellor in-memory)	60 s	Avoid repeated identical calls within one reasoning step
L2 (shared Redis)	5 min for TVL / 30 s for prices / 10 min for yields	Reads dominated by L2
L3 (Postgres warehouse)	1 h	Historical & analytical queries

For an 8-agent fleet doing 10 reasoning loops/hour, allocate per-Counsellor sub-budgets (e.g., 7,500 req/hour each) and enforce them with a token-bucket middleware in front of the REST client. Monitor `/usage/APIKEY` daily to avoid the 1 M/month cliff.

6.4 Knowledge-Graph / RAG Patterns

Persist DefiLlama responses as **structured facts** keyed by `(entity_slug, metric, timestamp)`:

```
fact(aave-v3, tvl_usd, 2026-05-25T00:00:00Z) = 15_200_000_000
fact(usdc, supply_ethereum, 2026-05-25T00:00:00Z) = 62_400_000_000
fact(uniswap, fees_24h_usd, 2026-05-25T00:00:00Z) = 4_120_000
```

This shape lets any Counsellor do RAG retrieval over historical state without re-hitting the API and enables explainable agent reasoning (every claim links back to a `(source, endpoint, timestamp)` triple). The DefiLlama skill `defi-data` is essentially a curated mapping of *natural-language questions* → *tool + params* — vendor it as part of your RAG corpus.

7. Security, Reliability, Operational

7.1 API Key Management

- Store the Pro key in your secrets manager (AWS Secrets Manager, HashiCorp Vault, sealed-secrets); **never** commit it. Note the key sits **in the URL path**, so be especially careful about log redaction — strip `/{{key}}/` from any captured URLs.
- Rotate quarterly; check usage at `/usage/APIKEY`.
- Per the official upgrade page (`pro.llama.fi`): "*DefiLlama contributors will have free 3 month access to premium API.*" File PRs to `DefiLlama-Adapters` for new protocols to subsidise the bill.

7.2 Handling Outages and Stale Data

- DefiLlama is a single point of failure for any system that builds on it. Mitigations:
 - Mirror critical endpoints (`/protocols`, `/v2/chains`, `/pools`) to your own warehouse with a 6-12 hour rolling window.
 - For prices, fail over to CoinGecko (DefiLlama already prices "almost all tokens using CoinGecko's API" — so this is the natural backstop). (`GitHub`)
 - Surface `stale_after` watermarks to your agents so they can avoid acting on data older than N hours.

7.3 Validating Swap Quotes Before Execution

Before sending a LlamaSwap-routed transaction:

1. **Sanity-check** `amountReturned` against your own price oracle — reject if it deviates by $> 2\%$ from `/prices/current/{token}` mid-market.
2. **Simulate on-chain** via `eth_call` against the router's `swap()` selector with the exact calldata returned by the adapter.
3. **Re-fetch the quote** within the last 30 seconds before submission; quotes expire.
4. **Cap slippage** at a hard maximum (e.g., 1% for stables, 3% for blue-chips, 5% for long-tail) — enforce on the contract `minOut` parameter, not just in the UI.

7.4 MEV and Sandwich Mitigation

- Prefer **CowSwap** (off-chain batch auctions, solver-protected) for size-sensitive trades — its `isMEVSafe` flag is `true`.
- For on-chain routes, use **private RPCs** (Flashbots Protect, MEV Blocker, Manifold) — LlamaSwap does *not* automatically use them; the user is responsible.
- Avoid round-number trade sizes that sandwich bots fingerprint.

7.5 TVL Methodology Quirks

DefiLlama publishes its methodology, but agents must understand the **known caveats** before quoting numbers to humans:

- **Tokens vesting in team contracts are excluded** from TVL — "we don't count any tokens that are not circulating or are yet to be issued." [GitHub](#)
- **Native staking for chain security is excluded** from chain TVL; liquid-staking protocols are tracked separately. [GitHub](#)
- **Bridge TVL is reported as an independent protocol** and does **not** contribute to any chain's headline TVL. [GitHub](#)
- **Restaking double-count toggle** (EigenLayer era): by default counted at the lowest layer (Lido) and *excluded* from EigenLayer headline TVL; users can toggle "include restaking."
- **Receipt tokens are not double-counted** within the same protocol. [GitHub](#)
- **Price source**: "almost all tokens are priced using CoinGecko's API," with on-chain Uniswap-V2 pool-weight pricing as a fallback for illiquid tokens. [GitHub](#)

Agents should attach a disclaimer like *"TVL per DefiLlama methodology; excludes vested/non-circulating tokens and native PoS stake"* whenever quoting headline numbers to non-technical principals.

8. Concrete Code Examples

8.1 Top 50 Protocols by TVL (TypeScript)

typescript

```
import axios from 'axios';

interface Protocol { name: string; tvl: number; chains: string[]; category: string; }

async function top50ByTvl(): Promise<Protocol[]> {
  const { data } = await axios.get<Protocol[]>('https://api.llama.fi/protocols', {
    return data.sort((a, b) => (b.tvl ?? 0) - (a.tvl ?? 0)).slice(0, 50);
  }

  top50ByTvl().then(rows => console.table(rows.map(r => ({ name: r.name, tvl: r.tvl.to
```

8.2 Top Yield Pools, Filtered (Python)

python

```
import httpx
from typing import Iterable

def top_yields(chain: str = "Ethereum", min_tvl_usd: float = 1e7, min_apy: float = 6,
               symbol: str | None = None) -> Iterable[dict]:
    resp = httpx.get("https://yields.llama.fi/pools", timeout=30.0)
    resp.raise_for_status()
    pools = resp.json()["data"]
    f = [p for p in pools
         if p.get("chain") == chain
         and (p.get("tvlUsd") or 0) >= min_tvl_usd
         and (p.get("apy") or 0) >= min_apy
         and (symbol is None or symbol.upper() in (p.get("symbol") or "").upper())]
    return sorted(f, key=lambda p: p["apyMean30d"], reverse=True)[:25]

for p in top_yields(symbol="USDC"):
    print(f"{p['project']}:18} {p['symbol']}:20} APY {p['apy']:6.2f}% TVL ${p['tvlUsd']:,}
```

8.3 Stablecoin Supply Stream (Python)

```
python

import httpx, time

def stream_stablecoin_supply(poll_sec: int = 3600):
    while True:
        try:
            data = httpx.get("https://stablecoins.llama.fi/stablecoins", timeout=30)
            for s in data["peggedAssets"][:10]:
                print(f"{s['symbol']:8} circulating={s['circulating'].get('peggedUSD')}")
        except Exception as e:
            print(f"err: {e}")
        time.sleep(poll_sec)
```

8.4 Calling the Official MCP Endpoint Programmatically (Python)

```
python

# Minimal MCP client using the streamable-http transport.
# Production code should use the official 'mcp' Python SDK.
import httpx, json, uuid

MCP_URL = "https://mcp.defillama.com/mcp"
BEARER = "<oauth_token_after_login_flow>" # obtained via OAuth, not raw API key

def call_tool(name: str, args: dict) -> dict:
    payload = {
        "jsonrpc": "2.0", "id": str(uuid.uuid4()), "method": "tools/call",
        "params": {"name": name, "arguments": args},
    }
    r = httpx.post(MCP_URL, json=payload,
                  headers={"Authorization": f"Bearer {BEARER}",
                          "Accept": "application/json, text/event-stream"},
                  timeout=60.0)
    r.raise_for_status()
    return r.json()["result"]

print(call_tool("get_market_totals", {}))
print(call_tool("get_yield_pools", {"chain": "Ethereum", "symbol": "USDC"}))
```

8.5 Conceptual LlamaSwap-Style Quote + Tx Build (TypeScript)

typescript

```
// Conceptual – adapters wrap underlying aggregator HTTP APIs.
type Quote = { adapter: string; amountReturned: bigint; estimatedGas: number;
  rawQuote: any; tokenApprovalAddress: string };

const adapters = ['1inch', '0x', 'paraswap', 'kyberswap', 'odos', 'openocean', 'cowswap'];

async function quoteAll(chain: string, from: string, to: string, amount: string,
  user: string, slippage: number): Promise<Quote[]> {
  const settled = await Promise.allSettled(
    adapters.map(a => quoteOne(a, chain, from, to, amount, { userAddress: user, slippage: slippage }));
  );
  return settled.flatMap(s => s.status === 'fulfilled' ? [s.value] : []);
}

function bestRoute(quotes: Quote[], gasPriceWei: bigint, gasPriceUsd: number): Quote {
  return quotes
    .map(q => ({ q, score: Number(q.amountReturned) - q.estimatedGas * Number(gasPriceUsd) }));
  .sort((a, b) => b.score - a.score)[0].q;
}

async function executeSwap(q: Quote, wallet: any) {
  // 1. Ensure ERC-20 approval for q.tokenApprovalAddress
  // 2. Call adapter.swap() – for on-chain aggregators this returns {to, data, value}
  //    For CowSwap this returns an off-chain order id.
  const tx = await adapterSwap(q.adapter, q.rawQuote, wallet.address);
  return wallet.sendTransaction(tx);
}
```

The adapter-interface shape this code assumes is **the same interface used in `LlamaSwap/interface`**, documented in §4.4 — so dropping in a real adapter module gives a working backend router.

9. Bibliography (Authoritative Sources)

Official Documentation

- **API docs** — <https://api-docs.defillama.com/>
- **LLM-friendly docs:** <https://api-docs.defillama.com/llms-free.txt>, <https://api-docs.defillama.com/llms-pro.txt>
- **OpenAPI specs:** <https://api-docs.defillama.com/defillama-openapi-free.json>, <https://api-docs.defillama.com/defillama-openapi-pro.json>

- **Pricing & API plan:** (<https://defillama.com/subscription>), (<https://docs.llama.fi/pro-api>), (<https://pro.llama.fi/>)
- **Adapter listing docs:** (<https://docs.llama.fi/list-your-project/other-dashboards>)
- **Methodology:** (<https://github.com/DefiLlama/docs>)
- **MCP product page:** (<https://defillama.com/mcp>)
- **MCP setup skill:** (<https://raw.githubusercontent.com/DefiLlama/defillama-skills/refs/heads/master/defillama-setup/SKILL.md>)

Official GitHub Repositories

- (github.com/DefiLlama/defillama-app) — frontend, GPL-3.0-or-later
- (github.com/DefiLlama/defillama-server) — backend
- (github.com/DefiLlama/DefiLlama-Adapters) — TVL adapters
- (github.com/DefiLlama/dimension-adapters) — fees/volumes/aggregators/options/derivatives adapters
- (github.com/DefiLlama/yield-server) — yields engine
- (github.com/DefiLlama/peggedassets-server) — stablecoins engine
- (github.com/DefiLlama/bridges-server) — bridges engine
- (github.com/DefiLlama/rpc-proxy) — LlamaRPC backend
- (github.com/DefiLlama/chainlist) — EVM network list, GPL-3.0
- (github.com/DefiLlama/icons) — protocol icon assets
- (github.com/DefiLlama/api-sdk) — official TS SDK
- (github.com/DefiLlama/python-sdk) — official Python SDK
- (github.com/DefiLlama/defillama-skills) — agent workflow skills
- (github.com/DefiLlama/api-docs) — public docs source
- (github.com/LlamaSwap/interface) — LlamaSwap frontend (no SPDX license file)

Community MCP Servers

- (github.com/dcSpark/mcp-server-defillama)
- (github.com/IQAIcon/mcp-defillama)
- (github.com/nic0xflamel/defillama-mcp)
- (github.com/demcp/demcp-defillama-mcp)
- (pulsesmcp.com/servers/pipeworx-defillama)

Community SDKs / References

- pypi.org/project/DeFiLlama/ — [itzmestar/DeFiLlama](https://github.com/itzmestar/DeFiLlama) (Apache 2.0)
- github.com/AstrolabDAO/swapper — backend port of the LlamaSwap routing pattern
- github.com/bgd-labs/llamaswap-interface — BGD Labs fork for embedding

Social / News

- **X / Twitter:** [@DeFiLlama](https://twitter.com/DeFiLlama) (rate-limit announcement: x.com/DeFiLlama/status/1609963521722257416)
 - **DL News editorial:** dlnews.com
 - **Discord:** discord.swap.defillama.com
 - **Founder:** [@0xngmi](https://twitter.com/0xngmi) on X / DefiLlama Discord
-

Recommendations (Staged Adoption)

Stage 1 (Week 1) — Direct REST + caching

- Stand up a Redis-fronted REST client against api.llama.fi. Implement: [/protocols](https://api.llama.fi/protocols), [/v2/chains](https://api.llama.fi/v2/chains), [/pools](https://api.llama.fi/pools), [/overview/fees](https://api.llama.fi/overview/fees), [/prices/current/*](https://api.llama.fi/prices/current/*), [/stablecoins](https://api.llama.fi/stablecoins).
- TTLs: TVL 10 min, prices 60 s, yields 15 min.
- Wrap with the official [@defillama/api](https://twitter.com/defillama/api) or [defillama-sdk](https://github.com/defillama/sdk) for typed access.
- **Threshold to advance:** > 400 RPM sustained (i.e., approaching the 500 RPM free-tier ceiling) → upgrade to the API plan (\$300/mo, 1,000 RPM, 1M calls/month) and add Pro endpoints (hacks, raises, unlocks, treasuries, derivatives, bridges-detail).

Stage 2 (Week 2) — MCP for Agent Counsellors

- Subscribe to the API plan.
- Wire each Counsellor to the official MCP at <https://mcp.defillama.com/mcp> with a curated tool subset per §6.1.
- Install the [defillama-skills](https://github.com/defillama-skills) workflow library on the CEO agent to standardise multi-tool reasoning patterns.
- **Threshold to advance:** agents start needing trade execution → Stage 3.

Stage 3 (Week 3-4) — Read-only LlamaSwap Embedding

- Embed the LlamaSwap iframe in your operator console for human-supervised execution.

- Add domain validation, mandatory slippage caps (1% stables / 3% blue-chips / 5% long-tail), and pre-trade quote sanity-check against `/prices/current/*`.
- **Threshold to advance:** human-in-the-loop becomes a bottleneck → Stage 4.

Stage 4 (Month 2) — Backend Router (Pattern D)

- Vendor either `AstrolabDAO/swapper` or write an equivalent against the adapter interface in §4.4.
- Run quote fan-out server-side, execute through your own RPC (Flashbots Protect or MEV Blocker for size).
- Mandatory: pre-execution `eth_call` simulation, on-chain `min0ut` enforcement, post-trade reconciliation against the DefiLlama price feed.
- Engage `@0xngmi` for an official LlamaSwap API key if you exceed the underlying aggregators' free quotas.

Stage 5 (Month 3+) — Hybrid Autonomy (Pattern E)

- Connect Counsellor agents' analytics output to the CEO's execution authority via a structured intent format.
- Require two-Counsellor sign-off on any trade > \$X (Risk + Treasury, e.g.).
- Mirror critical DefiLlama data to your own warehouse for outage resilience and explainability.

Caveats

1. **The Pro subscription and the API plan are different products and priced differently.** Per `docs.llama.fi/pro-api`, the **Pro tier (LlamaAI + dashboards)** is "\$49/month or \$490/year" with a "free 7-day trial available"; the **API plan (developer access)** is \$300/month with 1,000 RPM and 1M calls/month. *"API access is not included in the Pro plan."* Confirm before procurement. `llama` `GitHub`
2. **LlamaSwap's repository has no SPDX license file.** Vendors or commercial redistribution should be cleared with 0xngmi directly. The iframe widget is the safer integration path.
3. **The MCP server binds one DefiLlama account to one MCP client at a time** — a non-trivial constraint for multi-tenant agent fleets. The workaround is to procure separate API plans per persistent agent persona or to route all agent traffic through one MCP proxy that re-uses the single OAuth token.
4. **The full list of LlamaSwap adapters could not be definitively enumerated** from the GitHub tree page; the list in §4.3 is assembled from direct file fetches, merged/pending PRs,

and external coverage. Verify the current set by clicking through

`src/components/Aggregator/adapters/` in the GitHub UI.

5. **Rate-limit ceilings: the free tier is 500 RPM per IP** (officially announced by `@DefiLlama` on January 2, 2023); the **API tier is 1,000 RPM and 1M calls/month** per the official `docs.llama.fi/pro-api` page, with \$0.60 per 1,000 calls overage. Treat these as the working ceilings and design back-off accordingly.
6. **TVL is a measurement, not a fact.** DefiLlama is methodology-driven; restaking, wrapped assets, and liquid-staking tokens all have explicit accounting rules that occasionally change. Subscribe to the DefiLlama Twitter for methodology change announcements and cache the docs version alongside the data.
7. **DefiLlama is a single point of failure.** For any system whose financial decisions depend on its data, plan a CoinGecko-or-equivalent fallback for prices and a warehouse mirror for the rest.